

Python: отладка, логирование и тестирование

Лекция о применении pdb, модуля logging, видах тестирования и базовых приёмах написания тестов в pytest.

Структура материала

1. PDB как встроенный отладчик Python
2. Пошаговая отладка и анализ состояния
3. Логирование через logging
4. Виды тестирования и их роль
5. Pytest и структура тестов

PDB как встроенный отладчик Python

PDB — стандартный отладчик Python, который позволяет остановить выполнение программы и исследовать её состояние из командной строки. Он особенно полезен там, где нет графической IDE или нужно быстро локализовать проблему в консольном сервисе.

Базовый сценарий прост: в код добавляют `pdb.set_trace()`, после чего выполнение останавливается в нужной точке. Далее можно смотреть значения переменных, переходить по шагам, заходить внутрь вызовов и продолжать выполнение до следующего интересующего места.

Даже если в повседневной работе используется IDE, понимание PDB важно. Большая часть графических отладчиков опирается на те же базовые механизмы и ту же модель выполнения программы.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- `set_trace()` ставит программную точку останова в конкретном месте кода.
- `next` выполняет следующую строку без захода в вызываемую функцию.
- `step` позволяет зайти внутрь вызова и смотреть исполнение глубже.
- `continue` продолжает работу программы до следующей точки останова.
- `post_mortem` помогает анализировать уже произошедшее исключение.

Простейшая точка останова внутри функции

```
import pdb

def divide(a: float, b: float) -> float:
    pdb.set_trace()
    return a / b
```

Пошаговая отладка и анализ состояния

Отладка полезна не только для поиска явной ошибки, но и для понимания того, как программа реально проходит по ветвлениям и циклам. Часто баг проявляется не в момент падения, а намного раньше — когда переменная уже получила неверное значение.

Поэтому во время отладки стоит формулировать гипотезу: какое состояние программы считается корректным в каждой контрольной точке. Тогда PDB превращается не в хаотичный просмотр всех переменных, а в инструмент проверки инвариантов.

Для сложных случаев помогает посмертная отладка. Если программа упала с исключением, можно открыть стек вызовов именно в месте ошибки и изучить локальное окружение без повторного запуска всей сессии.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Отладка эффективнее, когда заранее понятен ожидаемый инвариант состояния.
- Лучше смотреть не все переменные подряд, а только те, что влияют на гипотезу о баге.
- Посмертная отладка полезна для редких и трудно воспроизводимых исключений.
- Комбинация отладчика и логов часто даёт лучший результат, чем любой из инструментов по отдельности.

Логирование через logging

Модуль logging нужен для систематической фиксации событий во время работы программы. В отличие от print, логирование поддерживает уровни важности, форматирование, вывод в разные назначения и централизованную настройку поведения.

Стандартные уровни помогают разделить сообщения по смыслу: DEBUG полезен для подробной диагностики, INFO описывает нормальный ход работы, WARNING отмечает потенциальные проблемы, ERROR и CRITICAL фиксируют реальные сбои разной тяжести.

Хорошее логирование показывает бизнес-событие и контекст: какой запрос пришёл, какой объект обрабатывался, что именно пошло не так. Логи не должны быть просто шумом из случайных строк.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- DEBUG подходит для технических подробностей и диагностики.
- INFO описывает штатные действия приложения.
- WARNING сигнализирует о ситуации, которая пока не сломала сценарий, но заслуживает внимания.
- ERROR и CRITICAL фиксируют сбои и критические нарушения работы.

Минимальная настройка логирования

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
)

logger = logging.getLogger(__name__)
logger.info("Application started")
```

Виды тестирования и их роль

Тестирование полезно рассматривать как набор уровней, а не как один общий термин. Модульные тесты проверяют маленький фрагмент кода, интеграционные тесты — взаимодействие нескольких частей, а системные и функциональные тесты смотрят на сценарий целиком.

У каждого уровня своя цель. Модульный тест быстро ловит ошибку в локальной логике, интеграционный показывает, как сочетаются слои и зависимости, а системный помогает убедиться, что продукт ведёт себя корректно в окружении, близком к реальному.

Хорошая тестовая стратегия не пытается решить всё одним видом тестов. Она распределяет проверки так, чтобы быстрые тесты ловили большую часть дефектов раньше, а более тяжёлые сценарии страховали ключевые пользовательские потоки.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Модульный тест проверяет небольшой и изолированный фрагмент логики.
- Интеграционный тест проверяет совместную работу нескольких компонентов.
- Функциональный тест валидирует конкретную пользовательскую функцию.
- Системный тест смотрит на приложение целиком в близком к реальности окружении.

Pytest и структура тестов

Pytest популярен благодаря простому синтаксису, мощной системе фикстур и удобному выводу результатов. Даже короткий тест получается читаемым: достаточно описать входные данные, вызвать функцию и сравнить результат с ожиданием.

Полезная мыслительная рамка — Arrange, Act, Assert. Сначала подготавливают данные и окружение, затем выполняют проверяемое действие, а после этого формулируют точное утверждение о результате.

Для учебных проектов этого уже достаточно, чтобы перейти от 'я проверил руками' к воспроизводимой автоматической проверке. В дальнейшем pytest легко расширяется параметризацией, фикстурами и тестами на исключения.

КЛЮЧЕВЫЕ ТЕЗИСЫ

- Arrange — подготовить входные данные и зависимости.
- Act — выполнить тестируемое действие.
- Assert — сформулировать ожидаемый результат одной или несколькими проверками.
- Тест должен описывать поведение кода, а не только воспроизводить его реализацию.

Минимальный тест в pytest

```
def add(a: int, b: int) -> int:
    return a + b

def test_add_returns_sum():
    assert add(2, 3) == 5
```

Материал подготовлен в обезличенном виде на основе локальных учебных заметок и приведён к единому формату для SMHUB.